

LECTURE 2

The Database Approach

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The distinction between data and metadata, and why a database needs both
- The full database environment: hardware, software, data, procedures, and people
- The roles people play around a database — DBA, designers, developers, end users
- The major components a DBMS is built from
- The core functions every DBMS performs
- A first look at data dependence vs. data independence

Data and Metadata

Data is the actual facts being stored. On its own, a raw value like 4500000 tells you nothing — a database needs a second layer describing the first: **metadata** (the system catalog / data dictionary), data about the data.

Data vs. metadata, for a bank's Account table

Data

“SL21”, 4500000, “B003” — the actual stored values

Metadata

accountNo is CHAR(5) and a primary key; balance is DECIMAL and must be ≥ 0 ; branchNo is a foreign key to Branch

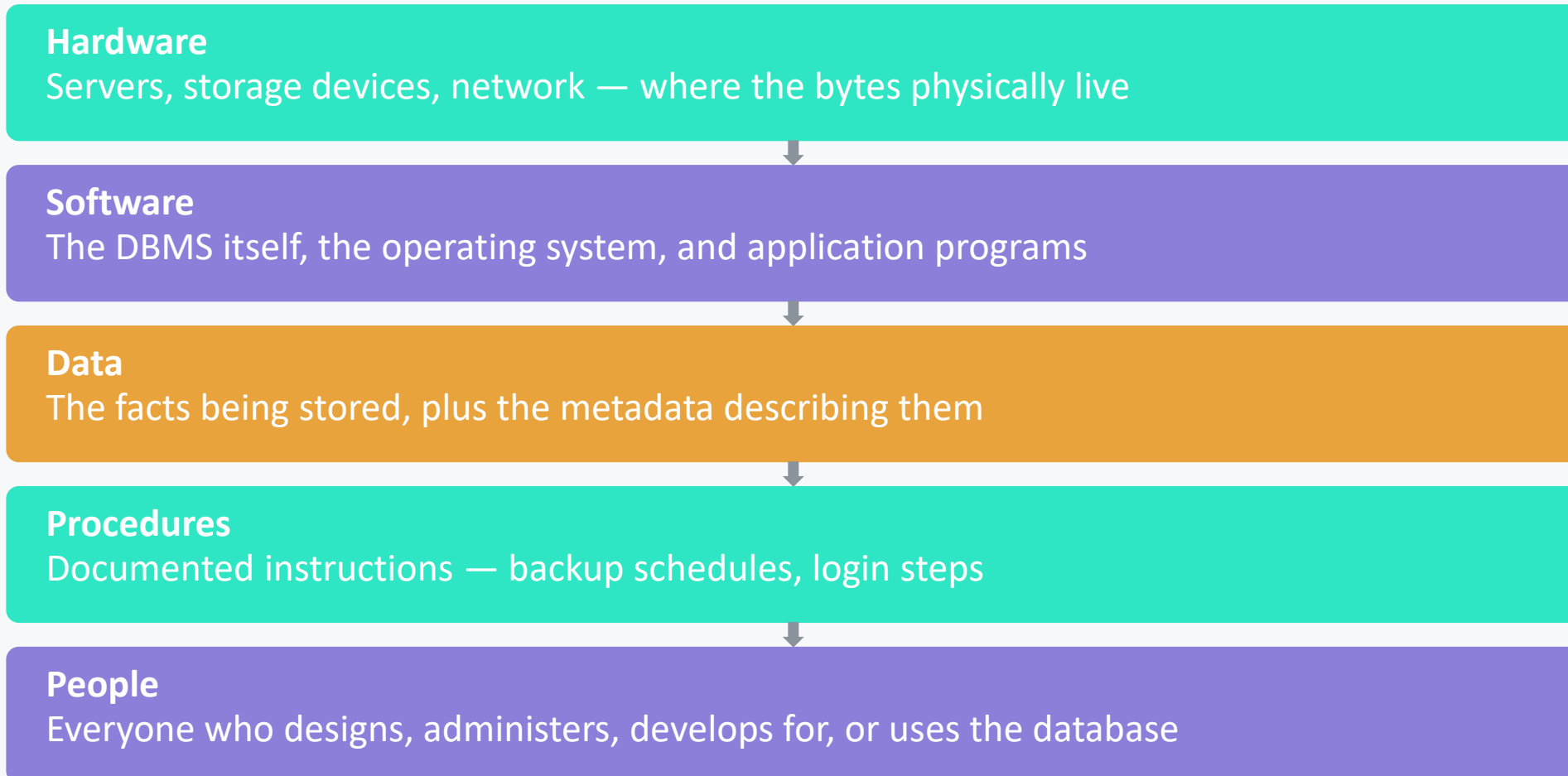
Metadata is why a DBMS can reject bad requests before running them. When an application tries to insert a letter into balance, the DBMS looks up balance's type in the catalog, sees it's numeric, and refuses the insert — general rule enforcement, not hand-coded per column.

Worked Example: Metadata Catching a Bad Insert

- An application tries to insert the text “N/A” into the Account table's balance column
- Before touching any data, the DBMS looks up balance in the catalog
- The catalog says balance is DECIMAL and must be ≥ 0 — “N/A” satisfies neither rule
- The DBMS refuses the insert immediately, with no special-case code written for balance specifically
- The same catalog lookup protects every column, in every table, automatically — that is general rule enforcement
- Without metadata, the DBMS would have no way to know balance was ever supposed to be numeric

The Database Environment

A database is five interacting components, per Connolly & Begg:



When One Component Fails: A Worked Scenario

- Hardware fails — the bank's database server goes down; even perfect table design can't serve a single query
- Software fails — the DBMS crashes or the OS needs patching; the data exists but nothing can reach it
- Data fails — metadata is wrong or missing, so bad values slip in silently
- Procedures fail — no documented backup schedule exists; a disk failure becomes permanent data loss
- People fail — the on-call DBA was never trained on the recovery tool; the outage runs for hours instead of minutes

The Database Environment, Applied to COMSATS

- Hardware — the servers in COMSATS's data center that physically hold the Student table
- Software — the DBMS plus the registrar's, library's, and finance's application programs
- Data — the Student relation itself, plus the metadata describing RegNo, Name, Address, Program
- Procedures — documented steps for enrolling a new student or restoring from last night's backup
- People — the DBA who manages the shared database, and the registrar, library, and finance staff who use it

Roles Around a Database

Role	Primary concern	Example task at a bank
Database Administrator (DBA)	Security, performance, backup/recovery, availability	Granting a new teller SELECT-only access to Account
Database Designers	Deciding what data to store and how it relates	Deciding Account needs a branchNo foreign key
Application Developers	Writing the programs end users interact with	Building the teller's “process withdrawal” screen
End Users	Using the finished application to do their job	A teller processing a customer's withdrawal

Naive vs. Sophisticated End Users

- Naive users interact only through a fixed application interface, unaware a database even exists underneath
- Example: a bank teller using a withdrawal screen — they click buttons and never see a table or write a query
- Sophisticated users write their own queries directly against the database, without a pre-built application
- Example: a risk analyst querying total outstanding loans per branch, built from scratch for a one-off question
- Most end users are naive by design — a well-built application means nobody needs to be sophisticated just to do their job

Why the DBA Role Matters

- In a file-based world, no single person was responsible for an organization's data as a whole
- Each application programmer managed only their own files, with their own ad hoc rules
- The database approach concentrates that responsibility into one role: the DBA
- Centralizing the data means centralizing the risk if it is mismanaged — one bad decision now affects everyone
- That is why the DBA's concerns — security, performance, backup/recovery, availability — cover the whole organization at once

Components of a DBMS

Zooming in from the environment to the DBMS software itself, several cooperating components each handle a distinct concern:

Query Processor

Parses and optimizes queries submitted by users and applications

Database Manager

Coordinates access to stored data; enforces integrity and security rules

Transaction Manager

Ensures concurrent operations stay correct and recoverable

File Manager

Manages the allocation of physical disk space for stored files

DML / DDL Precompiler

Translates database statements embedded in application code

Data Dictionary / Catalog

Stores the metadata every other component consults

A single `UPDATE Account SET balance = balance - 5000 WHERE accountNo = 'SL21'` passes through the query processor, the catalog, the transaction manager, and finally the file manager to write the new value to disk.

Worked Example: Tracing One Query Through the DBMS

Component	What It Does Here	Result
Query Processor	Parses the UPDATE statement and checks its syntax	Statement is valid and ready to execute
Data Dictionary / Catalog	Confirms Account and balance exist, and the update is permitted	Request is recognized as legal
Transaction Manager	Makes the change safely alongside other concurrent activity	Update is isolated and recoverable
File Manager	Writes the new balance value to disk	Change is physically stored

The query: UPDATE Account SET balance = balance - 5000 WHERE accountNo = 'SL21' — one statement, four components, each handling a distinct concern.

Functions of a DBMS (1/2)

- **Data storage, retrieval, and update** — the most basic function: store a value, get it back, change it
- **A user-accessible catalog** — metadata is itself queryable by tools and administrators
- **Transaction support** — a group of related operations either all complete or none do
- **Concurrency control** — correctness guarantees when many users access data at once
- **Recovery services** — restoring correct state after a hardware or software failure

Functions of a DBMS (2/2)

- **Authorization services** — controlling who is allowed to do what, down to individual tables or columns
- **Support for data communication** — integrating with the network software applications use to reach the database
- **Integrity services** — enforcing that stored data satisfies the organization's business rules
- **Services to promote data independence** — insulating applications from physical storage details
- **Utility services** — import/export, performance monitoring, and analysis tools

Functions in Action: A Withdrawal End to End

- Transaction support — debit the account and log the withdrawal as one unit; both happen or neither does
- Concurrency control — two withdrawals hitting the same account at once can't both succeed and overdraw it
- Integrity services — a rule rejects the withdrawal outright if it would push balance below zero
- Authorization services — the teller's login can debit accounts but cannot change the bank's interest rate table
- Recovery services — if the server crashes mid-withdrawal, the DBMS restores a correct state with no half-finished transaction left behind

Data Dependence vs. Data Independence — A First Look

File-based systems suffered from program-data dependence: application code was written assuming an exact physical file layout. Data independence is the DBMS's answer.

Data dependence (file-based)

Application code hard-codes field order and size; any physical change breaks every program that reads the file

Data independence (database approach)

Applications see a stable logical view; the DBMS absorbs physical changes underneath it

Don't Confuse Independence with “Never Changes”

- Data independence does not mean the database's structure is frozen
- It means changes at one level can be made without cascading into forced changes at another level
- The schema still evolves — new columns get added, storage gets reorganized, indexes get built and dropped
- What independence buys you is containment: a DBA's change doesn't force every application programmer to rewrite their code
- Lecture 4 splits this into logical and physical data independence, once the ANSI-SPARC three-schema architecture is on the table

From Data to a Working DBMS — Recap

- We opened the database up: data needs metadata beside it, or a raw value like 4500000 means nothing
- A working database is five components together — hardware, software, data, procedures, and people
- Distinct roles divide responsibility that file-based systems left scattered across every application programmer
- A DBMS's cooperating components together deliver its core functions — storage, transactions, concurrency, recovery, security, integrity
- Data independence is the database's direct answer to Lecture 1's program-data dependence problem

Key Takeaways

- Metadata (the system catalog) describes the data and lets a DBMS enforce rules generically rather than through hand-coded logic
- The database environment is hardware, software, data, procedures, and people together — excellent table design can still fail if procedures or people are neglected
- Distinct roles — DBA, database designers, developers, and end users (naive and sophisticated) — divide responsibility for a shared database
- A DBMS is built from cooperating components (query processor, database manager, transaction manager, file manager, catalog) delivering storage, transactions, concurrency, recovery, security, integrity, and data independence
- Data independence lets one level change without forcing dependent applications to change — expanded fully in Lecture 4

Looking Ahead: Lecture 3

- Lecture 3 looks at the languages people and programs actually use to talk to a DBMS
- You'll meet DDL (defining structure), DML (querying and updating data), and DCL (controlling access)
- Every role from this lecture — the DBA, the designer, the developer, the end user — reaches the database through one of these languages
- Today's components and functions are the machinery; Lecture 3 is the interface everyone actually touches